
PrivaSecOps

Private Cloud Platform
with GitOps & DevSecOps

Documentation technique complète

Mr. Ayoub ELMORTAJI

Élève-Ingénieur Cybersécurité & Cloud Computing
ENSAM Casablanca • 2026



K3S



Open Policy Agent



GitHub Actions



vmware®



Terraform



Table des matières

- 1. Introduction et contexte
 - 2. Architecture globale
 - 3. Phase 1 — Cloud privé OpenStack + k3s
 - 4. Phase 2 — GitOps avec ArgoCD
 - 5. Phase 3 — Pipeline DevSecOps
 - 6. Phase 4 — Sécurité runtime avec Falco
 - 7. Phase 5 — Observabilité (Prometheus + Grafana)
 - 8. Phase 6 — Infrastructure as Code (Terraform + Ansible)
 - 9. Résultats et démonstration
 - 10. Conclusion et perspectives
- Annexes

1. Introduction et contexte

1.1 Contexte du projet

Les entreprises modernes, en particulier les opérateurs télécoms comme Inwi, doivent gérer des infrastructures cloud complexes tout en garantissant la sécurité à chaque étape du cycle de vie des applications. Le défi est de construire une plateforme qui automatise le déploiement, intègre la sécurité de bout en bout, et fournit une observabilité complète — le tout sur un cloud privé maîtrisé.

Ce projet répond à ce défi en combinant les meilleures pratiques de Cloud Computing, GitOps et DevSecOps dans une plateforme intégrée.

1.2 Objectifs

- Déployer un cloud privé complet avec OpenStack
- Orchestrer des conteneurs avec Kubernetes (k3s)
- Implémenter le GitOps avec ArgoCD pour le déploiement continu
- Intégrer la sécurité à chaque étape du pipeline (Shift-Left Security)
- Détecter les menaces en temps réel avec Falco
- Monitorer l'ensemble avec Prometheus et Grafana
- Automatiser l'infrastructure avec Terraform et Ansible (IaC)

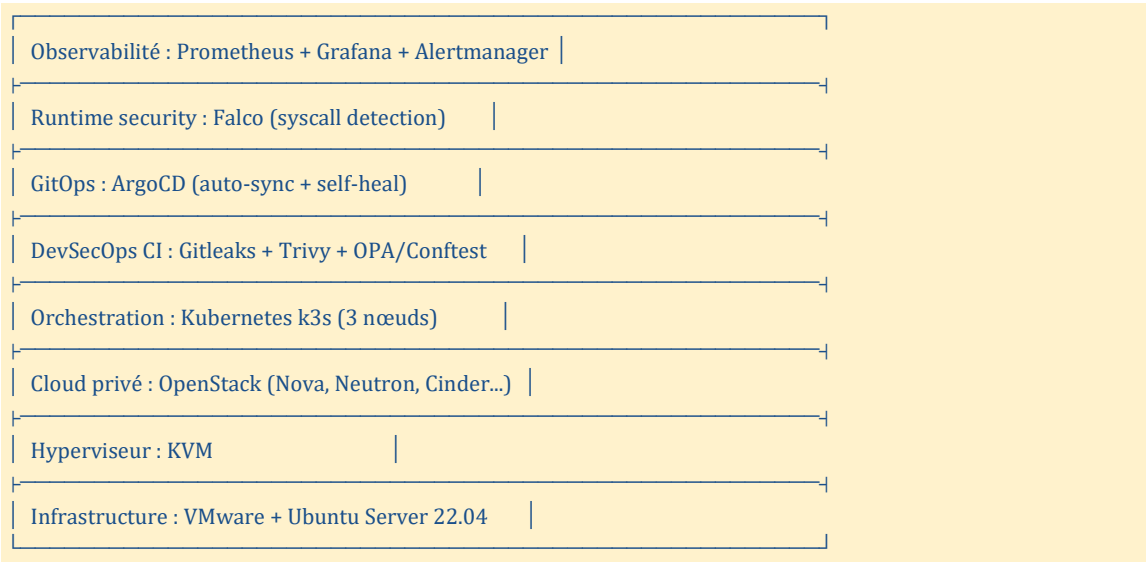
1.3 Technologies utilisées

Couche	Outil	Rôle
Cloud privé	OpenStack (DevStack)	IaaS — compute, network, storage
Hyperviseur	KVM	Virtualisation des VMs
Orchestration	Kubernetes (k3s)	Gestion des conteneurs
GitOps	ArgoCD	Déploiement continu depuis Git
Package manager	Helm	Packaging des apps K8s
CI/CD	GitHub Actions	Pipeline automatique
Secrets scan	Gitleaks	Détection de credentials dans le code
Manifest scan	Trivy	Détection de misconfigurations K8s
Policy as code	OPA / Conftest	Validation de politiques K8s
Runtime security	Falco	Détection de menaces syscall-level
Métriques	Prometheus	Collecte de métriques
Dashboards	Grafana	Visualisation
Alerting	Alertmanager	Notifications
IaC	Terraform	Provisionnement infrastructure
Configuration	Ansible	Configuration automatique

2. Architecture globale

2.1 Vue d'ensemble

Le projet est structuré en couches empilées, chacune apportant une fonctionnalité spécifique. L'architecture suit le modèle cloud-native avec une séparation claire entre l'infrastructure (OpenStack), l'orchestration (Kubernetes), le déploiement (ArgoCD) et la sécurité (DevSecOps + Falco).



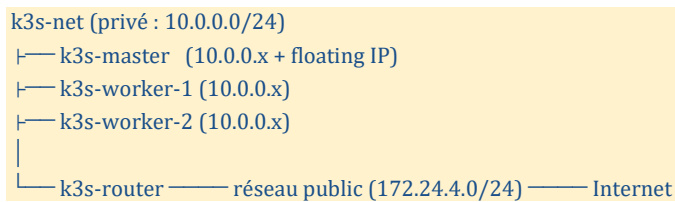
2.2 Infrastructure physique

Composant	Spécification
Machine hôte	Intel i5, 24 GB RAM, 245 GB SSD
Hyperviseur hôte	VMware Workstation (nested virt.)
VM DevStack	Ubuntu 22.04, 16 GB RAM, 4 vCPU, 150 GB
Virtualisation imbriquée	KVM activé (Intel VT-x)

2.3 Cluster Kubernetes

Nœud	Rôle	vCPU	RAM	Disk	IP privée
k3s-master	Control plane	2	3 GB	30 GB	10.0.0.x
k3s-worker-1	Worker	2	2 GB	20 GB	10.0.0.x
k3s-worker-2	Worker	2	2 GB	20 GB	10.0.0.x

2.4 Réseau



Le routeur virtuel Neutron connecte le réseau privé au réseau public via NAT. Seul le master possède une floating IP pour l'accès SSH externe. Les workers sont accessibles uniquement depuis le master (bastion pattern).

2.5 Flux de déploiement complet

1. Développeur pousse du code sur GitHub
2. GitHub Actions déclenche le pipeline CI
 - └─ Gitleaks : scan de secrets
 - └─ Trivy : scan de misconfigurations K8s
 - └─ OPA/Conftest : validation de politiques
3. Si tous les gates passent → merge autorisé
4. ArgoCD détecte le changement dans Git
5. ArgoCD applique automatiquement sur k3s
6. Kubernetes fait un rolling update
7. Falco surveille l'exécution en temps réel
8. Prometheus collecte les métriques
9. Grafana affiche les dashboards
10. Alertmanager notifie en cas d'anomalie

3. Phase 1 — Cloud privé OpenStack + k3s

3.1 Installation de DevStack

DevStack est un script qui installe tous les services OpenStack sur une seule machine pour le développement et l'apprentissage. Les services installés sont : Keystone (identité), Nova (compute), Neutron (réseau), Glance (images), Cinder (stockage), et Horizon (dashboard).

Configuration (local.conf)

```
[[local|localrc]]
ADMIN_PASSWORD=admin123
DATABASE_PASSWORD=admin123
RABBIT_PASSWORD=admin123
SERVICE_PASSWORD=admin123
HOST_IP=192.168.227.132
FLAT_INTERFACE=ens33
LIBVIRT_TYPE=kvm
Q_AGENT=openvswitch
Q_ML2_PLUGIN_MECHANISM_DRIVERS=openvswitch
Q_ML2_TENANT_NETWORK_TYPE=vxlan
NEUTRON_CORE_PLUGIN=ml2
ENABLED_SERVICES=rabbit,mysql,key
ENABLED_SERVICES+=,placement-api,placement-client
ENABLED_SERVICES+=,n-api,n-cond,n-sch,n-cpu,n-novnc,n-api-meta
ENABLED_SERVICES+=,g-api,g-reg
ENABLED_SERVICES+=,q-svc,q-agt,q-dhcp,q-l3,q-meta,neutron
ENABLED_SERVICES+=,c-api,c-vol,c-sch
ENABLED_SERVICES+=,horizon
```

3.2 Création de l'infrastructure OpenStack

Réseau

```
openstack network create k3s-net
openstack subnet create k3s-subnet \
  --network k3s-net --subnet-range 10.0.0.0/24 --dns-nameserver 8.8.8.8
openstack router create k3s-router
openstack router set k3s-router --external-gateway public
openstack router add subnet k3s-router k3s-subnet
```

Security group

```
openstack security group create k3s-sg
openstack security group rule create k3s-sg --protocol tcp --dst-port 22
openstack security group rule create k3s-sg --protocol tcp --dst-port 6443
openstack security group rule create k3s-sg --protocol udp --dst-port 8472
openstack security group rule create k3s-sg --protocol tcp --dst-port 10250
openstack security group rule create k3s-sg --protocol tcp --dst-port 30000:32767
openstack security group rule create k3s-sg --protocol any --remote-group k3s-sg
openstack security group rule create k3s-sg --protocol icmp
```

Port	Proto	Usage
22	TCP	SSH
6443	TCP	API Kubernetes

8472	UDP	Flannel VXLAN (réseau inter-nœuds)
10250	TCP	Kubelet
30000-32767	TCP	NodePort (exposition des apps)
ICMP	-	Ping

Flavors et VMs

```
openstack flavor create k3s-master --vcpus 2 --ram 3072 --disk 30
openstack flavor create k3s-worker --vcpus 2 --ram 2048 --disk 20

openstack server create k3s-master \
  --image jammy-server-cloudimg-amd64 --flavor k3s-master \
  --network k3s-net --security-group k3s-sg --key-name k3s-keypair \
  --user-data userdata.yaml --wait
```

3.3 Installation de k3s

Master

```
curl -sL https://get.k3s.io | sh -s - server \
  --disable traefik --write-kubeconfig-mode 644
```

Workers

```
curl -sL https://get.k3s.io | K3S_URL=https://<MASTER_IP>:6443 \
  K3S_TOKEN="<TOKEN>" sh -
```

Résultat

```
kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
k3s-master    Ready    control-plane   9m   v1.36.3+k3s1
k3s-worker-1  Ready    <none>        1m   v1.36.3+k3s1
k3s-worker-2  Ready    <none>        9s   v1.36.3+k3s1
```

4. Phase 2 — GitOps avec ArgoCD

4.1 Principe GitOps

GitOps est une méthodologie où Git est la seule source de vérité pour l'infrastructure et les applications. Un agent (ArgoCD) surveille un repo Git en permanence et garantit que l'état du cluster correspond toujours à ce qui est défini dans Git.

- **Modèle Pull** : ArgoCD tire les changements depuis Git (pas de push kubectl)
- **Déclaratif** : on décrit l'état désiré, pas les étapes pour y arriver
- **Auto-sync** : chaque changement dans Git est appliqué automatiquement
- **Self-heal** : si quelqu'un modifie le cluster manuellement, ArgoCD remet l'état de Git

4.2 Installation d'ArgoCD

```
kubectl create namespace argocd
kubectl apply -n argocd -f \
  https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

# Exposition via NodePort
kubectl patch svc argocd-server -n argocd \
  -p '{"spec": {"type": "NodePort"}}'
```

4.3 Structure du repo Git

```
k8s-config/
├── .github/
│   ├── workflows/
│   └── security.yml    ← pipeline CI/CD
├── apps/
│   ├── nginx/
│   │   ├── deployment.yaml ← manifests K8s
│   │   └── service.yaml
│   └── argocd/
│       ├── root-app.yaml    ← app-of-apps
│       └── README.md
```

4.4 Définition d'une Application ArgoCD

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: nginx-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/AyoubElmortaji/k8s-config.git
    path: apps/nginx
    targetRevision: main
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
```

```
automated:  
  prune: true  
  selfHeal: true
```

4.5 Tests GitOps réalisés

Test	Action	Résultat
Auto-sync	Modifier replicas dans Git (3→5)	5 pods en 30 secondes sans kubectl
Self-heal	Supprimer un pod manuellement	ArgoCD recrée le pod immédiatement
Rollback	git revert du changement	Retour à l'état précédent automatique

5. Phase 3 — Pipeline DevSecOps

5.1 Shift-Left Security

Le Shift-Left Security consiste à intégrer les contrôles de sécurité le plus tôt possible dans le cycle de développement — dès le premier commit, pas avant la mise en production. Chaque push sur Git déclenche automatiquement 3 gates de sécurité.

5.2 Les 3 security gates

Gate	Outil	Ce qu'il vérifie	Action si échec
Secrets scan	Gitleaks	Mots de passe, API keys dans le code	Merge bloqué
Manifest scan	Trivy	Misconfigurations dans les YAML K8s	Merge bloqué
Policy check	OPA/Conftest	Resource limits, tag latest interdit	Merge bloqué

5.3 Pipeline GitHub Actions

```
name: DevSecOps Pipeline
"on":
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  gitleaks:
    name: Secrets Scan
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: gitleaks/gitleaks-action@v2

  trivy:
    name: Manifest Scan
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aquasecurity/trivy-action@master
      with:
        scan-type: config
        scan-ref: apps/
        severity: CRITICAL,HIGH
        exit-code: 1

  opa:
    name: Policy Check
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
- name: Run OPA Conftest
run: conftest test apps/ -p policy/
```

5.4 Politiques OPA

```
package main
import rego.v1

deny contains msg if {
  input.kind == "Deployment"
  not input.spec.template.spec.containers[0].resources.limits
  msg := "Deployment must have resource limits"
}

deny contains msg if {
  input.kind == "Deployment"
  some container in input.spec.template.spec.containers
  endswith(container.image, ":latest")
  msg := "Image tag latest is not allowed"
}
```

5.5 Deployment sécurisé (hardened)

Suite aux détections de Trivy, le deployment nginx a été durci avec un securityContext complet :

```
securityContext:
  runAsNonRoot: true
  seccompProfile:
    type: RuntimeDefault
containers:
- name: nginx
  securityContext:
    allowPrivilegeEscalation: false
    readOnlyRootFilesystem: true
    runAsNonRoot: true
  capabilities:
    drop: [ALL]
```

5.6 Résultat du pipeline

Gate	Statut	Détails
Secrets Scan (Gitleaks)	✅ Passed	Aucun credential détecté
Manifest Scan (Trivy)	✅ Passed	0 misconfiguration après hardening
Policy Check (OPA)	✅ Passed	Resource limits présents, pas de tag latest

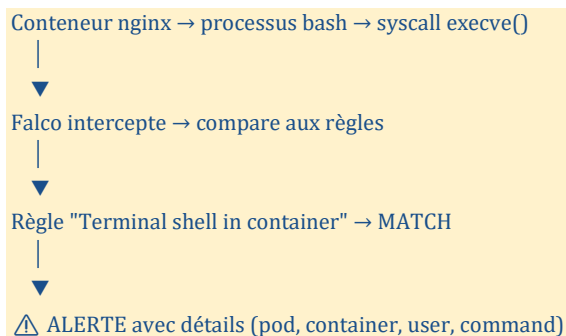
6. Phase 4 — Sécurité runtime avec Falco

6.1 Pourquoi Falco

Le pipeline DevSecOps vérifie la sécurité AVANT le déploiement. Mais une fois qu'une application tourne dans le cluster, qui surveille ce qui se passe à l'intérieur des conteneurs ? Falco comble ce gap en surveillant les appels système (syscalls) en temps réel.

6.2 Comment Falco fonctionne

Falco intercepte les syscalls du noyau Linux et les compare à des règles de détection. Chaque action dans un conteneur (ouvrir un fichier, lancer un processus, établir une connexion réseau) génère des syscalls que Falco analyse.



6.3 Installation

```
helm repo add falcosecurity https://falcosecurity.github.io/charts
helm install falco falcosecurity/falco \
  --namespace falco --create-namespace \
  --set falcosidekick.enabled=true \
  --set falcosidekick.webui.enabled=true
```

6.4 Test de détection

Simulation d'une attaque : ouverture d'un shell dans un conteneur nginx.

```
kubectrl exec -it <nginx-pod> -- sh
# Commandes suspectes :
whoami
cat /etc/passwd
```

Alerte Falco détectée immédiatement :

```
Notice: A shell was spawned in a container
container_name: nginx
k8s_pod_name: nginx-55d999779-4hlfn
user: root
command: sh
rule: Terminal shell in container
MITRE ATT&CK: T1059 (Execution)
```

7. Phase 5 — Observabilité

7.1 Les 3 piliers de l'observabilité

Pilier	Outil	Ce qu'il fournit
Métriques	Prometheus	CPU, RAM, réseau, nombre de pods, requêtes/sec
Logs	Loki (prévu)	Logs centralisés de tous les services
Dashboards	Grafana	Visualisation en temps réel de tout

7.2 Installation

```
helm repo add prometheus-community \
https://prometheus-community.github.io/helm-charts
helm install monitoring prometheus-community/kube-prometheus-stack \
--namespace monitoring --create-namespace \
--set grafana.service.type=NodePort
```

7.3 Composants installés

- Prometheus : scrape les métriques de tous les nœuds et pods toutes les 15 secondes
- Grafana : dashboards pré-configurés pour Kubernetes (CPU, RAM, réseau par nœud/namespace/pod)
- Alertmanager : gère les alertes et les envoie vers email, Slack ou Telegram
- Node Exporter : expose les métriques système de chaque nœud (DaemonSet)
- Kube State Metrics : expose l'état des objets K8s (deployments, pods, services)

7.4 Dashboards disponibles

- Kubernetes / Compute Resources / Cluster — vue globale du cluster
- Kubernetes / Compute Resources / Namespace (Pods) — métriques par pod
- Node Exporter / Nodes — CPU, RAM, disk, réseau par nœud
- Kubernetes / API Server — performance de l'API K8s

7.5 Accès aux dashboards

Dashboard	Tunnel SSH	URL locale
ArgoCD	ssh -L 9090:master:31903 stack@devstack	https://localhost:9090
Grafana	ssh -L 3000:master:32419 stack@devstack	http://localhost:3000
Horizon	Direct	http://192.168.227.132/dashboard

8. Phase 6 — Infrastructure as Code (Terraform + Ansible)

8.1 Objectif

Tout ce qui a été créé manuellement dans les phases précédentes (réseau OpenStack, VMs, installation k3s) peut être automatisé avec Terraform et Ansible. L'objectif est de pouvoir détruire et recréer toute l'infrastructure avec deux commandes.

8.2 Terraform — Provisionnement de l'infrastructure

Qu'est-ce que Terraform ?

Terraform est un outil d'Infrastructure as Code (IaC) qui permet de définir l'infrastructure de façon déclarative dans des fichiers .tf. Il communique avec les APIs d'OpenStack (et AWS, Azure, GCP...) pour créer les ressources automatiquement.

- Déclaratif : tu décris l'état final désiré, pas les étapes
- Idempotent : exécuter 2 fois la même config ne change rien si l'infra existe déjà
- State file : Terraform garde un fichier d'état (terraform.tfstate) pour savoir ce qui existe
- Plan : terraform plan montre ce qui va être créé/modifié/supprimé avant d'agir

Structure du projet Terraform

```
terraform/  
├── main.tf      ← ressources principales  
├── variables.tf ← variables paramétrables  
├── outputs.tf   ← valeurs de sortie (IPs, etc.)  
├── provider.tf  ← configuration OpenStack  
└── terraform.tfvars ← valeurs des variables
```

Provider OpenStack (provider.tf)

```
terraform {  
  required_providers {  
    openstack = {  
      source = "terraform-provider-openstack/openstack"  
      version = "~> 3.0"  
    }  
  }  
}  
  
provider "openstack" {  
  auth_url   = "http://192.168.227.132/identity"  
  user_name  = "admin"  
  password   = "admin123"  
  tenant_name = "admin"  
  domain_name = "Default"  
}
```

Ressources principales (main.tf)

```
# Réseau privé  
resource "openstack_networking_network_v2" "k3s_net" {  
  name      = "k3s-net"  
  admin_state_up = true
```

```

}

resource "openstack_networking_subnet_v2" "k3s_subnet" {
  name      = "k3s-subnet"
  network_id = openstack_networking_network_v2.k3s_net.id
  cidr      = "10.0.0.0/24"
  dns_nameservers = ["8.8.8.8"]
}

# Routeur
resource "openstack_networking_router_v2" "k3s_router" {
  name      = "k3s-router"
  external_network_id = data.openstack_networking_network_v2.public.id
}

resource "openstack_networking_router_interface_v2" "k3s_ri" {
  router_id = openstack_networking_router_v2.k3s_router.id
  subnet_id = openstack_networking_subnet_v2.k3s_subnet.id
}

# Security group
resource "openstack_networking_secgroup_v2" "k3s_sg" {
  name = "k3s-sg"
}

resource "openstack_networking_secgroup_rule_v2" "ssh" {
  security_group_id = openstack_networking_secgroup_v2.k3s_sg.id
  direction        = "ingress"
  protocol          = "tcp"
  port_range_min    = 22
  port_range_max    = 22
  remote_ip_prefix  = "0.0.0.0/0"
}

# VM Master
resource "openstack_compute_instance_v2" "k3s_master" {
  name      = "k3s-master"
  image_name = "jammy-server-cloudimg-amd64"
  flavor_name = "k3s-master"
  key_pair   = "k3s-keypair"
  security_groups = ["k3s-sg"]
  user_data   = file("userdata.yaml")
  network {
    name = openstack_networking_network_v2.k3s_net.name
  }
}

# Floating IP
resource "openstack_networking_floatingip_v2" "master_fip" {
  pool = "public"
}

resource "openstack_compute_floatingip_associate_v2" "master_fip" {
  floating_ip = openstack_networking_floatingip_v2.master_fip.address
  instance_id = openstack_compute_instance_v2.k3s_master.id
}

```

```
}
```

Commandes Terraform

```
terraform init    # initialise les providers
terraform plan    # montre ce qui va être créé
terraform apply   # crée l'infrastructure
terraform destroy # détruit tout
```

8.3 Ansible — Configuration automatique

Qu'est-ce qu'Ansible ?

Ansible est un outil d'automatisation de configuration. Là où Terraform crée les VMs, Ansible les configure : installer k3s, configurer le système, déployer des services.

- Agentless : pas besoin d'installer quoi que ce soit sur les serveurs cibles (utilise SSH)
- Idempotent : exécuter 2 fois le même playbook ne change rien
- Playbooks : fichiers YAML qui décrivent les tâches à exécuter
- Inventaire : liste des serveurs cibles et leurs groupes

Structure du projet Ansible

```
ansible/
├── inventory.yml    ← liste des serveurs
├── playbook-k3s.yml ← installation k3s
├── playbook-harden.yml ← hardening OS
├── roles/
│   ├── k3s-master/
│   │   └── tasks/main.yml
│   ├── k3s-worker/
│   │   └── tasks/main.yml
```

Inventaire (inventory.yml)

```
all:
  children:
    masters:
      hosts:
        k3s-master:
          ansible_host: "{{ master_floating_ip }}"
          ansible_user: ubuntu
          ansible_ssh_private_key_file: ~/.ssh/k3s-key
    workers:
      hosts:
        k3s-worker-1:
          ansible_host: 10.0.0.x
          ansible_user: ubuntu
          ansible_ssh_private_key_file: ~/.ssh/k3s-key
          ansible_ssh_common_args: "-J ubuntu@{{ master_floating_ip }}"
        k3s-worker-2:
          ansible_host: 10.0.0.x
          ansible_user: ubuntu
          ansible_ssh_private_key_file: ~/.ssh/k3s-key
          ansible_ssh_common_args: "-J ubuntu@{{ master_floating_ip }}"
```

Playbook — installation k3s (playbook-k3s.yml)

```
---
- name: Install k3s master
  hosts: masters
  become: true
  tasks:
    - name: Install k3s server
      shell: >
      curl -sL https://get.k3s.io | sh -s - server
      --disable traefik --write-kubeconfig-mode 644
    args:
      creates: /usr/local/bin/k3s

    - name: Get node token
      slurp:
        src: /var/lib/rancher/k3s/server/node-token
      register: node_token

    - name: Set token fact
      set_fact:
        k3s_token: "{{ node_token.content | b64decode | trim }}"

- name: Join k3s workers
  hosts: workers
  become: true
  tasks:
    - name: Install k3s agent
      shell: >
      curl -sL https://get.k3s.io |
        K3S_URL=https://{{ hostvars["k3s-master"].ansible_host }}:6443
        K3S_TOKEN={{ hostvars["k3s-master"].k3s_token }}
        sh -
    args:
      creates: /usr/local/bin/k3s-agent
```

Playbook — hardening OS (playbook-harden.yml)

```
---
- name: Harden all nodes
  hosts: all
  become: true
  tasks:
    - name: Update system
      apt:
        update_cache: yes
        upgrade: dist

    - name: Disable root SSH login
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: "^PermitRootLogin"
        line: "PermitRootLogin no"
      notify: restart sshd

    - name: Disable password authentication
```

```
lineinfile:
  path: /etc/ssh/sshd_config
  regexp: "^PasswordAuthentication"
  line: "PasswordAuthentication no"
  notify: restart sshd

- name: Install fail2ban
  apt:
    name: fail2ban
    state: present

handlers:
- name: restart sshd
  service:
    name: sshd
    state: restarted
```

Commandes Ansible

```
ansible-playbook -i inventory.yml playbook-harden.yml
ansible-playbook -i inventory.yml playbook-k3s.yml
```

8.4 Workflow IaC complet

```
# 1. Créer l'infrastructure
cd terraform/
terraform init
terraform apply -auto-approve

# 2. Configurer les VMs
cd ../ansible/
ansible-playbook -i inventory.yml playbook-harden.yml
ansible-playbook -i inventory.yml playbook-k3s.yml

# Résultat : cluster k3s complet en ~10 minutes
# Sans toucher une seule commande manuellement
```

9. Résultats et démonstration

9.1 Scénario de démonstration

La démonstration end-to-end montre le pipeline complet en action :

Étape 1 — Développeur modifie le code

```
# Modifier replicas de 3 à 5 dans deployment.yaml
git commit -m "Scale nginx to 5 replicas"
git push
```

Étape 2 — Pipeline CI se déclenche

GitHub Actions :

- ☒ Gitleaks → pas de secrets
- ☒ Trivy → manifests sécurisés
- ☒ OPA/Conftest → politiques respectées

Étape 3 — ArgoCD déploie

ArgoCD détecte le changement → auto-sync
kubectl get pods → 5 pods Running (au lieu de 3)

Étape 4 — Simulation d'attaque

```
kubectrl exec -it <pod> -- sh
```

→ Falco alerte : "Terminal shell in container"
→ MITRE ATT&CK : T1059 (Execution)

Étape 5 — Observabilité

Grafana dashboard → CPU, RAM, pods en temps réel
Alertmanager → notification envoyée

9.2 Métriques du projet

Métrique	Valeur
Nœuds Kubernetes	3 (1 master + 2 workers)
Services OpenStack utilisés	6 (Keystone, Nova, Neutron, Glance, Cinder, Horizon)
Security gates CI/CD	3 (Gitleaks, Trivy, OPA)
Temps de déploiement via GitOps	< 30 secondes
Temps de détection Falco	< 1 seconde
Dashboards Grafana	4+ pré-configurés
Outils de sécurité intégrés	5 (Gitleaks, Trivy, OPA, Falco, securityContext)

10. Conclusion et perspectives

10.1 Réalisations

Ce projet a permis de concevoir et déployer une plateforme cloud privée complète intégrant les meilleures pratiques DevSecOps. La sécurité est intégrée à chaque étape : avant le déploiement (Shift-Left avec Gitleaks, Trivy, OPA), pendant le déploiement (GitOps avec ArgoCD), et après le déploiement (Falco pour la détection runtime).

L'observabilité avec Prometheus et Grafana fournit une visibilité complète sur l'état du cluster et des applications, permettant une détection rapide des anomalies.

10.2 Compétences acquises

- Cloud privé : OpenStack (Nova, Neutron, Cinder, Glance, Keystone, Horizon)
- Orchestration : Kubernetes (k3s), kubectl, Helm
- GitOps : ArgoCD, auto-sync, self-heal, app-of-apps pattern
- DevSecOps : Gitleaks, Trivy, OPA/Conftest, GitHub Actions
- Runtime security : Falco (syscall-level detection, MITRE ATT&CK)
- Observabilité : Prometheus, Grafana, Alertmanager
- IaC : Terraform (OpenStack provider), Ansible (playbooks, inventaire)
- Linux avancé : networking, iptables, systemd, SSH tunnels
- Debugging : diagnostiquer et résoudre des problèmes dans un système distribué

10.3 Perspectives et améliorations

- Firewall : déployer pfSense ou Check Point CloudGuard comme VM OpenStack devant le cluster
- HashiCorp Vault : gestion centralisée des secrets avec injection automatique dans K8s
- Loki : agrégation de logs centralisée (Falco → Loki → Grafana)
- Network Policies : micro-segmentation avec Calico ou Cilium
- OPA Gatekeeper : admission controller K8s pour appliquer les politiques au niveau API server
- Scan d'images Docker : intégrer Trivy image scan dans le pipeline CI
- Multi-environnements : dev, staging, prod avec branches Git séparées
- Haute disponibilité : k3s multi-master avec embedded etcd

Annexes

A. Commandes OpenStack utilisées

```
source openrc admin admin
openstack token issue
openstack network create / list
openstack subnet create
openstack router create / set / add subnet
openstack security group create / rule create
openstack flavor create
openstack keypair create
openstack server create / list / start / stop / delete
openstack floating ip create
openstack server add floating ip
openstack console log show
```

B. Commandes Kubernetes utilisées

```
kubectl get nodes / pods / svc / deployments
kubectl get pods -n <namespace>
kubectl get pods -o wide
kubectl apply -f <file>
kubectl create namespace <name>
kubectl create deployment <name> --image=<img> --replicas=N
kubectl expose deployment <name> --type=NodePort --port=80
kubectl delete deployment / svc
kubectl exec -it <pod> -- sh
kubectl logs <pod>
kubectl patch svc <name> -p '...'
kubectl port-forward svc/<name> <port>:<port>
```

C. Commandes Helm utilisées

```
helm repo add <name> <url>
helm repo update
helm install <release> <chart> --namespace <ns> --create-namespace
helm list -A
helm uninstall <release> -n <ns>
```

D. Checklist de redémarrage

```
# Après reboot de la VM DevStack :
cd /opt/stack/devstack && ./rejoin-stack.sh
source openrc admin admin
sudo ip addr add 172.24.4.1/24 dev br-ex 2>/dev/null
sudo ip link set br-ex up
sudo sysctl -w net.ipv4.ip_forward=1
sudo iptables -t nat -A POSTROUTING -s 172.24.4.0/24 -o ens33 -j MASQUERADE
openstack server start k3s-master k3s-worker-1 k3s-worker-2
```

E. Liens du projet

- GitHub : <https://github.com/AyoubElmortaji/k8s-config>
- ArgoCD Dashboard : <https://localhost:9090> (via SSH tunnel)
- Grafana Dashboard : <http://localhost:3000> (via SSH tunnel)
- Horizon Dashboard : <http://192.168.227.132/dashboard>